

UNIVERSITÀ
POLITECNICA
DELLE MARCHE

Corso di «Ingegneria del Software»

A.A. 2024/2025

Parte IX: Il software come processo

Prof. Domenico Ursino
d.ursino@univpm.it

Verifica, validazione e test – Introduzione

- Queste tre attività sono spesso considerate insieme con il termine VV&T.
- La verifica è il processo con cui ci si assicura di aver costruito il prodotto correttamente, cioè di aver soddisfatto i requisiti documentati.
- La validazione si occupa di dimostrare che è stato costruito il prodotto giusto, ovvero che il sistema svolge efficacemente il suo compito, anche da punti di vista che non sono stati espressi nei requisiti.
- La tecnica principe per assicurare entrambe le cose è di provare il sistema, ovvero di svolgere il test.

Verifica, validazione e test – Revisione di qualità

- La definizione ad alto livello di qualità del software è legata al soddisfacimento dei requisiti del cliente.
- Per assicurare ciò è possibile richiedere che il sistema includa caratteristiche che non interessano direttamente all'utente, ma che contribuiscono al raggiungimento degli obiettivi di suo interesse.
- Così si può richiedere che il sistema permetta di svolgere i test in modo efficace.
- Il sistema, inoltre, dovrà essere facilmente modificabile per rispondere velocemente a eventuali segnalazioni di errore o richieste di cambiamento.
- L'esperienza insegna a diffidare di chi sostiene che un software avrà vita breve; i programmi di successo durano molto più di quanto si pensi: è così che è nato il Millenium Bug.
- Nel software, come in tutti gli altri campi, è diventato comune usare il termine "qualità" come sinonimo di "alta qualità".

Verifica, validazione e test – Verifica

- La verifica richiede sempre di confrontare un artefatto ben preciso (il prodotto) con un altro (la specifica).
- L'esempio più comune di verifica è il test del codice rispetto ai requisiti.
- Nel contesto di uno sviluppo basato su UML è possibile considerare le seguenti fasi:
  - Verifica che i casi d'uso soddisfino i requisiti descritti nella relativa specifica.
  - Verifica che le classi siano in grado di fornire i casi d'uso.
  - Verifica che il codice corrisponda alle classi che compongono il progetto.
- In uno sviluppo iterativo questi controlli dovranno essere eseguiti in ogni iterazione.
- Nella verifica si includono anche alcuni "check di sanità", quali la compilabilità del codice e la sua esecuzione senza errori.
- Con quest'ultimo termine si intende un comportamento inaspettato che non è giustificabile da una circostanza eccezionale o da un errore da parte degli utenti.

Verifica, validazione e test – Verifica

- Un programma non deve mai dare errori di segmentazione, bloccarsi, segnalare che un messaggio non è stato compreso o esibire altre caratteristiche “evidentemente sbagliate”.
- I modelli UML che fanno parte del progetto devono essere sintatticamente corretti e coerenti.
- Al di là del test del codice, come è possibile svolgere questi check?
- L’approccio più informale consistere nel prendere i due oggetti, esaminarli e valutare nella propria mente se c’è una corrispondenza appropriata.
- L’estremo più formale, al contrario, potrebbe richiedere una dimostrazione matematica che i due oggetti siano equivalenti.
- UML non possiede una semantica formale per cui la verifica matematica di un modello UML è impossibile in via di principio.

Verifica, validazione e test – Verifica

- Le verifiche formali richiedono moltissimo tempo e sono costose, cosicché tendono a essere eseguite solo nel caso di sistemi che presentano aspetti critici di sicurezza.
- La tecnica di verifica più diffusa è la compilazione!
- Un buon tool di modellazione UML aiuta gli sviluppatori a evitare errori banali e a produrre modelli sintatticamente corretti; inoltre potrebbe supportare la verifica di coerenza tra diagrammi differenti.
- Alcuni strumenti sono in grado di generare codice eseguibile dai modelli UML e viceversa.
- Una regola di verifica importante è quella di affidare la verifica a un programmatore diverso da quello che ha sviluppato originariamente l'elemento in esame.
- Una lista dei punti critici aiuta a non dimenticare nessuna delle cause più comuni dei problemi.
- Può essere utile anche pianificare qualche tipo di revisione o ispezione del codice.

Verifica, validazione e test – Validazione

- La verifica è ben definita: sappiamo bene quello che stiamo verificando.
- La validazione è più ardua perché non possiamo dire esattamente cosa stiamo cercando: in pratica qualsiasi cosa renda il prodotto meno utile per gli utenti.
- Per eseguire una validazione in modo efficace occorre coinvolgere i clienti.
- I problemi e le incomprensioni devono essere verificati il prima possibile; questa è la ragione principale per cui si adotta un processo di sviluppo iterativo.
- Il modo più semplice per coinvolgere un cliente è fargli firmare documenti che attestino l'accettazione delle specifiche delle funzionalità fornite dal sistema nelle varie iterazioni.
- Non è facile valutare un sistema basandosi unicamente su una descrizione testuale.
- Più aumenta l'incertezza sui requisiti e più diventa necessario eseguire iterazioni rapide per ottenere un feedback in tempi brevi.
- All'inizio del progetto o di un'iterazione ad alto rischio potrebbe valere la pena realizzare un prototipo "da buttare".

Verifica, validazione e test – Validazione – Usabilità

- Un sistema di alta qualità deve anche fare in modo che lo si possa effettivamente utilizzare al meglio.
- Gli utenti dovranno essere in grado di capire subito come eseguire un'operazione, come svolgere le attività velocemente e senza sforzo e come recuperare da eventuali errori.
- Questo aspetto del software è spesso trascurato.
- Le ragioni principali sono due:
  - Se una persona capisce come si fa qualcosa è praticamente impossibile che possa mettersi nei panni di chi invece non lo capisce.
  - I programmatori non sono affatto esempi di utenti tipici.
- In definitiva è impossibile valutare da soli l'usabilità del proprio sistema e anche i colleghi possono aiutare solo marginalmente.

Verifica, validazione e test – Validazione – Usabilità

- Le altre due possibili risorse a cui appoggiarsi sono il parere di un esperto di usabilità e le prove con utenti reali.
- In un caso ideale si dovrebbero sfruttare entrambe in un ciclo iterativo.
- Una soluzione al problema dell'usabilità è la cosiddetta progettazione centrata sull'utente.
- Una piccola quantità di lavoro ben focalizzato può apportare grandi benefici.
  - Un singolo giorno di testing di usabilità può produrre un aumento del 25% nell'efficienza di un tipico sistema.

Verifica, validazione e test – Testing

- Il testing contribuisce sia alla verifica che alla validazione ma più probabilmente alla prima.
- I test possono essere eseguiti su diversi artefatti: sistemi in esecuzione, progetti e parti.
- Gli obiettivi principali sono tre:
  - Trovare i bug.
  - Convincere il cliente che non ci sono bug.
  - Fornire informazioni che aiuteranno l'evoluzione del sistema, ad esempio requisiti futuri, priorità degli utenti, dati concreti sulla performance.
- Il primo obiettivo naturalmente è quello più importante:

Un test di successo è quello che identifica un bug

Verifica, validazione e test – Testing

- È facile far si che il sistema passi tutti i test: basta usare una suite di test inadeguata.
- Eliminare tutti i difetti è difficile ma l'uso combinato di tecniche diverse può aiutare a identificare il maggior numero.
- Un elenco di possibili tipologie di test è il seguente:
  - I test di usabilità verificano che il sistema sia facile da usare in modo efficace.
  - I test di unità (o di modulo) esaminano i moduli (o le classi) del sistema uno per uno.
  - I test di integrazione verificano che le parti del sistema funzionino insieme nel modo corretto.
  - I test di sistema controllano che il sistema soddisfi i requisiti, sia funzionali che non funzionali.
  - Il test di accettazione viene eseguito dal cliente o da un gruppo indipendente che lo rappresenta; serve a convalidare che il sistema funzioni effettivamente come desiderato.

Verifica, validazione e test – Testing

- I test delle prestazioni possono aver luogo a qualsiasi livello e assicurano che la performance del sistema è, o sarà, soddisfacente.
- Lo stress testing mette il sistema sotto pressione con carichi di lavoro molto più alti di quelli attesi per verificare che la performance diminuisca gradatamente senza fallire catastroficamente.
- I test di regressione verificano che le caratteristiche del sistema che funzionavano correttamente prima di una modifica continuino a non dare problemi.
    - I test eseguiti sono i test di unità per i moduli interessati e di integrazione per i sottosistemi che includono unità modificate.
    - In caso di necessità si possono eseguire nuovamente i test di usabilità, di sistema, di accettazione, di prestazioni e di stress.

- Queste categorie non sono mutuamente esclusive e i loro confini non sono sempre chiari.
- L'aspetto che varia di più è l'estensione del test di integrazione.

Verifica, validazione e test – Testing – Selezionare ed eseguire i test

- I test possono essere generalmente suddivisi in:
  - Test a scatola nera, che guardano la specificità delle entità testate.
  - Test a scatola bianca, che guardano la loro struttura interna.
- Quali sono gli aspetti da testare?
- N.H. Petschenik ha identificato tre regole importanti per assicurare che il cliente percepisca il software come di alta qualità:
  - È più importante testare il comportamento dell'intero sistema che il comportamento dei suoi componenti.
  - È più importante verificare che il sistema continui a svolgere correttamente le funzioni già supportate in precedenza piuttosto che quelle introdotte da poco.
  - È più importante testare i casi tipici piuttosto che quelli che comprendono casi limite.

Verifica, validazione e test – Testing – Selezionare ed eseguire i test

- Dal momento che è necessario svolgere il testing di regressione, i test devono essere:
  - Ripetibili.
  - Documentati.
  - Precisi
  - Eseguiti su software dalla configurazione controllata.
- Può essere necessario automatizzare in qualche modo il processo di testing.
- Un semplice script può aiutare a svolgere le seguenti attività:
  - Eseguire un insieme prefissato di test.
  - Registrare quali test sono stati effettuati, con la relativa configurazione e i relativi risultati.
  - Confrontare i valori ottenuti con i risultati attesi.

Verifica, validazione e test – Testing – Selezionare ed eseguire i test

- L'automazione, però, non può compensare una cattiva pianificazione dei test.
- La specifica dei test per un artefatto, cioè la descrizione dei test da eseguire e i risultati attesi, deve essere eseguita "il prima possibile, ma non prima".
- Così facendo è più facile comprendere chiaramente i requisiti e assicurarsi che essi possano essere verificati in seguito.
- Alcune persone sostengono che i test di accettazione e i relativi risultati dovrebbero essere definiti precisamente all'inizio del progetto e che gli sviluppatori dovrebbero essere tenuti per contratto a produrre un sistema in grado di superarli.
- Il problema di questo approccio è che, per definire un test, è necessario conoscere quantomeno un abbozzo dei requisiti.
- L'identificazione dei requisiti attraverso i casi d'uso può aiutare ad accertarsi che i test siano scritti al giusto livello di astrazione.

Verifica, validazione e test – Testing – Selezionare ed eseguire i test

- Un buon modo di evitare una progettazione affrettata è concentrarsi sui cambiamenti che avvengono tra l'inizio e la fine di uno scenario, ad esempio stabilire quali sono i benefici per gli attori.
- Quando si adotta uno sviluppo iterativo è buona regola scrivere i test per l'iterazione all'inizio dell'iterazione stessa.
- È molto difficile descrivere appropriatamente test complessi, ma alcuni errori saranno rilevati solo in seguito ad una complessa sequenza di azioni.
- Un buon modo per procedere è il seguente:
  - Scrivere codice di qualità testato a livello di unità.
  - Eseguire il test del software con lo scopo esplicito di "rompere il sistema".
- La stesura di test brevi, precisi e ben documentati è assolutamente necessaria ma non è sufficiente.

Verifica, validazione e test – Testing – Selezionare ed eseguire i test

- Questo tipo di validazione è particolarmente importante per le interfacce grafiche che sono difficili da testare in modo sistematico.

- Senza uno strumento speciale è praticamente impossibile eseguire test automatici su una GUI e anche se lo si possiede non è affatto facile.

- Gran parte dei problemi di interfaccia riguarda in realtà l'usabilità (e i test di usabilità non possono essere automatizzati) oppure comportamenti che qualsiasi GUI deve fornire, indipendentemente dal sistema sottostante ("check di sanità").

- Ogni pulsante, ad esempio, deve avere un'etichetta o un altro indicatore visuale per indicare gli effetti di un click e la manipolazione dei controlli non deve mai bloccare l'applicazione o portarla al crash.

Verifica, validazione e test – Testing – Aspetti introdotti dall'orientamento ad oggetti

- Il paradigma object oriented introduce altri tre problemi principali a livello dei test di unità:
  - Qual è la minima unità di test.
  - L'incapsulamento.
  - L'ereditarietà e il polimorfismo.

Verifica, validazione e test – Testing – Aspetti introdotti dall'orientamento ad oggetti – Quale unità di test

- L'unità di test deve corrispondere quantomeno a una classe.
- La ragione è che un oggetto è dotato di uno stato da cui può dipendere il suo comportamento.
- I diagrammi di stato mettono in evidenza il problema ma suggeriscono anche un modo di affrontarlo.
  - Un oggetto deve essere quantomeno testato in ognuno dei suoi stati.
- È necessario sottoporre a test ogni singola transizione del diagramma.
- Le classi che hanno diagrammi di stato complessi sono difficili da comprendere e testare; è opportuno cercare di evitarle il più possibile nei progetti.

Verifica, validazione e test – Testing – Aspetti introdotti dall'orientamento ad oggetti – Incapsulamento

- L'incapsulamento è importante proprio perché riduce il rischio di introdurre errori a causa di una comprensione errata delle interazioni tra componenti.
- Questo può rendere difficile il test di un componente incapsulato.
- Una possibile soluzione è aggiungere in ogni classe un metodo che restituisce tutte le informazioni di stato e che dovrà essere usata unicamente per il testing.

Verifica, validazione e test – Testing – Aspetti introdotti dall'orientamento ad oggetti – Ereditarietà e Polim.

- Quando una sottoclasse estende una classe accuratamente testata ridefinendo con l'overriding alcuni dei suoi metodi, quali test devono essere eseguiti sulla nuova sottoclasse?
- Si potrebbe pensare che sia sufficiente verificare il funzionamento dei metodi ridefiniti; in realtà, a causa del binding dinamico, non è così.
- Per stare al sicuro è necessario testare tutte le funzionalità di una sottoclasse, anche quelle che sono state ereditate dalla superclasse senza alcuna modifica.
- Il polimorfismo basato sull'ereditarietà aggiunge un ulteriore livello di complessità.
  - Quando un oggetto della classe A collabora con un oggetto della classe B potrebbe in realtà interagire con un'istanza di qualsiasi sottoclasse di B che magari non era stata ancora inventata quando A ha superato i test.
- È opportuno usare l'ereditarietà solo quando i benefici superano gli svantaggi.

Verifica, validazione e test – Testing – Aspetti introdotti dall'orientamento ad oggetti – Ereditarietà e Polim.

- La dipendenza dallo stato è racchiusa all'interno degli oggetti che hanno interfacce ben definite per cui si può confidare di risparmiare fatica nei test di integrazione e di sistema e di uscirne con un bilancio positivo.

- Per questo comunque non è sufficiente adottare un approccio orientato agli oggetti: l'architettura del sistema ha un'importanza fondamentale.

Verifica, validazione e test – Testing – Perché i test sono eseguiti così spesso in modo inadeguato?

- Perché accade che sistemi difettosi passano continuamente i test e sono rilasciati agli utenti?
- Le ragioni di ciò sono molteplici:
  - Il testing è un'attività molto molto noiosa, specialmente per quanto riguarda la documentazione dell'intero processo.
    - Le attività noiose sono tollerabili se è chiaro a tutti che sono importanti ma possono risultare estremamente demoralizzanti se sono percepite come un dovere burocratico.
  - Il testing richiede molto tempo, quindi costa molto denaro (può anche superare il 50% del tempo allocato per il progetto – in media si assesta intorno al 30%).
    - L'alternativa, ovvero svolgere test inadeguati, può anche essere peggiore, ma il suo costo non è evidente fino al momento in cui cominciano ad arrivare le lamentele.
  - Spesso il testing è pianificato verso la fine del progetto ed è una delle attività più sacrificate quando le scadenze per il rilascio cominciano ad incombere.
  - I clienti potrebbero fare pressioni per rilasciare un prodotto non testato adeguatamente nella data prevista anziché ritardare il rilascio.

Verifica, validazione e test – Revisioni e ispezioni del codice

- Le revisioni e le ispezioni di vario genere possono contribuire sia alla verifica che alla validazione, anche se probabilmente sono più utili per quest'ultima attività.
- Una revisione, ispezione o walkthrough consiste essenzialmente in un incontro in cui si cercano di identificare i problemi di qualche prodotto del processo di sviluppo.
- Talvolta sono denominate collettivamente FTR (Formal Technical Review).
- L'Ispezione di Fagan è probabilmente quella più nota.
- Le FTR si differenziano molto per i dettagli e il grado di formalismo, ma solitamente condividono tutte le seguenti caratteristiche:
  - I partecipanti sono allo stesso livello gerarchico (non manager e subordinati).
  - L'artefatto da revisionare è fornito a tutti i partecipanti in anticipo.
  - L'incontro segue un'agenda predefinita e produce un rapporto dei difetti riscontrati, unito a un certo grado di analisi degli stessi.

Verifica, validazione e test – Revisioni e ispezioni del codice

  - Durante il meeting non si deve discutere su come porre rimedio ai difetti.
  - Viene definita una procedura successiva, che spesso coinvolge Autore e Moderatore, allo scopo di assicurare che i difetti siano stati gestiti adeguatamente.

- Ad esempio, la revisione di progetto relativamente informale di un modello delle classi UML per un'iterazione di sviluppo destinata a realizzare alcuni casi d'uso potrebbe impegnare un gruppo di 4-7 persone per un paio d'ore.
- In una revisione del codice, un processo simile viene applicato al codice sorgente.
- Le ispezioni di Fagan sono considerate utili da molti esperti per identificare i difetti, soprattutto quelli di progetto.
- Tuttavia, le FTR sono meno diffuse del testing e ancor meno nei progetti che adottano un moderno processo iterativo anziché un classico modello a cascata.

Verifica, validazione e test – Revisioni e ispezioni del codice

- I problemi delle FTR sembrano ricadere in due categorie principali:
  - Una FTR, il cui scopo principale è quello di identificare i difetti presenti in un artefatto, può essere facilmente percepita come un attacco rivolto a chi l'ha sviluppato.
  - Le FTR richiedono molto tempo e talvolta producono lunghe liste di difetti minori senza riuscire nel contempo a rilevare quelli più importanti.

Manutenzione ed evoluzione del software

- Siamo in un periodo relativamente strano nell'ingegneria del software in cui le attività di manutenzione ed evoluzione rappresentano gran parte, se non la maggior parte, dei costi del software, eppure la maggior parte della nostra comprensione è basata su studi che sono superati da decenni.
- Sommerville definisce la manutenzione del software come: «Il processo di modifica del software dopo la consegna quando la transizione dallo sviluppo alla soluzione non è senza soluzione di continuità».
- Più in generale, la manutenzione comporta la modifica di un programma dopo che è stato messo in uso.
- Di solito non ci aspettiamo che la manutenzione comporti grandi cambiamenti all'architettura del sistema.
- Piuttosto, i cambiamenti sono fatti modificando i componenti esistenti e aggiungendo nuovi componenti al sistema.
- Dal punto di vista di un programmatore, la questione chiave è che il programmatore deve capire il programma e la sua struttura.
- La manutenzione è importante perché il software è cruciale per il successo dell'azienda e perché il software è molto complicato.
- Nel mondo di oggi, la maggior parte del budget del software è dedicato alla modifica del software esistente piuttosto che allo sviluppo di un nuovo software.

Manutenzione ed evoluzione del software

- I prodotti software di successo sono di solito in uso molto più a lungo che in sviluppo, rendendo la manutenzione ancora più cruciale.
- Se il software non continua ad adattarsi ai cambiamenti dei bisogni e dell'ambiente, diventa progressivamente meno utile.
- I cambiamenti sono inevitabili e avvengono per diverse ragioni:
  - Nuovi requisiti emergono quando il software viene usato.
  - L'ambiente aziendale cambia.
  - I guasti devono essere riparati.
  - Nuovi computer o attrezzature vengono aggiunti al sistema.
  - La crescente base di utenti rende insufficienti le prestazioni o l'affidabilità.
- Alcuni cambiamenti sono dovuti allo stretto legame con l'ambiente: quando il software viene installato, cambia quell'ambiente, cambiando di fatto i requisiti del software.

Manutenzione ed evoluzione del software – Falsi miti sulla manutenzione ed evoluzione del software

- In pratica, molti ingegneri e manager del software sono stati falsamente convinti di diverse cose che semplicemente non sono vere.
- Alcuni di questi miti sono spiegati e contraddetti qui di seguito.
  - *Mito: Abbiamo già un libro che è pieno di standard e procedure per costruire software, questo non fornirà alla mia gente tutto quello che deve sapere?*
    - Realtà: Il libro degli standard può benissimo esistere, ma viene usato?
    - In molti casi, le risposte alle seguenti domande sono "no".
      - I professionisti del software sanno che esiste e dove trovarlo?
      - Riflette la moderna pratica di ingegneria del software?
      - È completo?
      - È semplificato per migliorare i tempi di consegna pur mantenendo l'attenzione sulla qualità?

Manutenzione ed evoluzione del software – Falsi miti sulla manutenzione ed evoluzione del software

- *Mito: Se siamo in ritardo, possiamo aggiungere altri programmatori e recuperare il ritardo.*
  - Realtà: Lo sviluppo del software non è un processo meccanicistico come la produzione.
  - Come ha detto Brooks: "aggiungere persone ad un progetto software in ritardo lo rende più tardivo".
- *Mito: Se esternalizzo il progetto software ad una terza parte, posso semplicemente rilassarmi e lasciare che quella ditta lo costruisca.*
  - Realtà: Se un'organizzazione non capisce come gestire e controllare i progetti software internamente, invariabilmente farà fatica quando esternalizza progetti software.
- *Mito: Gli obiettivi generali sono sufficienti per iniziare a programmare - possiamo riempire i dettagli più tardi.*
  - Realtà: Una scarsa definizione iniziale è una delle cause principali del fallimento degli sforzi di software. Se non si sa cosa si vuole all'inizio, non si otterrà ciò che si vuole.

Manutenzione ed evoluzione del software – Falsi miti sulla manutenzione ed evoluzione del software

- *Mito: I requisiti del progetto cambiano continuamente, ma il cambiamento può essere facilmente adattato perché il software è flessibile.*
  - Realtà: È vero che i requisiti del software cambiano, ma l'impatto di un cambiamento dipende da quando viene introdotto.
- *Mito: Una volta che scriviamo il programma e lo facciamo funzionare, il nostro lavoro è finito.*
  - Realtà: Qualcuno una volta ha detto che "prima cominci a 'scrivere codice', più tempo ti ci vorrà per finire".
  - I dati dell'industria indicano che tra il 60% e l'80% di tutto lo sforzo speso sul software sarà speso dopo che è stato consegnato al cliente.
- *Mito: Finché non faccio "girare" il programma non ho modo di valutarne la qualità.*
  - Realtà: Uno dei più efficaci meccanismi di assicurazione della qualità del software può essere applicato fin dall'inizio di un progetto - la revisione tecnica formale.
  - Le revisioni del software sono più efficaci dei test per trovare certe classi di difetti del software.

Manutenzione ed evoluzione del software – Falsi miti sulla manutenzione ed evoluzione del software

- *Mito: L'unico prodotto di lavoro consegnabile per un progetto di successo è il programma di lavoro.*
  - Realtà: Un programma di lavoro è solo una parte di una configurazione software che include molti elementi.
  - La documentazione fornisce una base per un'ingegneria di successo e, più importante, una guida per il supporto del software.
- *Mito: L'ingegneria del software ci farà creare documentazione voluminosa e non necessaria e ci rallenterà sempre.*
  - Realtà: L'ingegneria del software non riguarda la creazione di documenti.
  - Si tratta di creare qualità.
  - Una migliore qualità porta a rielaborazioni ridotte.
  - E una ridotta rielaborazione porta a tempi di consegna più rapidi.

Manutenzione ed evoluzione del software – Vari tipi di manutenzione

- La manutenzione è stata tradizionalmente divisa in quattro tipi.
- Le percentuali date nei libri di testo sono tutte simili, e basate su studi dei primi anni '80.
- Sembra improbabile che i numeri siano ancora validi, quindi bisogna prenderli con le pinze; sfortunatamente, sono tutto ciò che abbiamo.
  - Manutenzione perfettiva (50%): miglioramenti in cui nuove operazioni e perfezionamenti sono aggiunti alle funzioni esistenti.
  - Manutenzione adattativa (25%): modifica dell'applicazione per soddisfare nuove circostanze operative.
  - Manutenzione correttiva (21%): eliminazione degli errori nella funzionalità del programma.
  - Manutenzione preventiva (4%): modificare un programma per migliorare la sua futura manutenibilità.
- Alcuni autori considerano la manutenzione di emergenza come un tipo di manutenzione correttiva non programmata.

Manutenzione ed evoluzione del software – Vari tipi di manutenzione

- Un grande cambiamento negli ultimi 20 anni nel modo in cui il software viene mantenuto è che le aziende spesso rilasciano una versione intenzionalmente parziale, poi la perfezionano nel tempo.
- Non è chiaro se questo sia propriamente considerato perfettivo, adattivo o qualcos'altro.
- Ma è chiaro che il termine evoluzione è più appropriato di manutenzione.
- Questo è possibile quando il software è facile da aggiornare.
  - Per esempio, il software nei nostri dispositivi mobili può essere aggiornato ogni giorno, al contrario del software nei sottomarini, che normalmente può essere aggiornato solo quando il sottomarino visita il porto.
- In effetti, questo è uno dei principali punti a favore dell'Intenet of Things.
  - Risolvere un problema di software nella mia lavatrice richiede una visita costosa da parte di un tecnico per sostituire un circuito, mentre se la mia lavatrice è su internet, la società può scaricare un nuovo software quasi a costo zero.

Manutenzione ed evoluzione del software – Costi

- Modificare il software, non importa come o quando, è difficile e costoso.
- Quando software e hardware sono strettamente integrati, il software è anche spesso visto come la parte più facile da cambiare.
- Ma solo perché il costo di manutenzione è così lento, non significa che il costo di progettazione e implementazione delle modifiche sia gratuito.
- I grandi programmi software sono estremamente complicati e molto difficili da capire. Eppure devono essere capiti prima di poterli cambiare.
- Molti cambiamenti sono affrettati, quindi le modifiche sono spesso mal progettate e implementate.
- Per peggiorare le cose, cambiare il software quasi invariabilmente inietta nuovi difetti che devono essere riparati in seguito.
- Sommerville sostiene che il 90% di tutti i costi del software sono legati all'evoluzione del software.
- Anche se questo è esagerato, pochi dubitano che più della metà dei costi del software siano maturati dopo il primo deployment.

Manutenzione ed evoluzione del software – Costi

- I costi sono dovuti a fattori sia tecnici che non tecnici.
- La maggior parte degli ingegneri concorda sul fatto che il costo di fare lo stesso tipo di cambiamento aumenta man mano che il software invecchia.
- Questo è dovuto a molti fattori, tra cui la perdita di memoria sulla progettazione e costruzione iniziale, i cambiamenti nelle tecnologie e nei linguaggi, e la difficoltà di capire i cambiamenti precedenti (spesso sciatti).
- Alcuni tipici fattori di costo di natura tecnica sono i seguenti:
  - Complessità del codice: una struttura logica e di controllo complessa è difficile da capire e quindi difficile da cambiare.
  - Cambiamenti nei linguaggi di programmazione: se il codice da cambiare è scritto in un linguaggio che non è più comunemente usato, è difficile da cambiare.
    - Ancora più difficile è se il nuovo codice deve essere scritto in un linguaggio diverso.
  - Cambiamenti nell'infrastruttura software: se il software sottostante, il middleware o le librerie sono cambiati, allora i programmatori devono capire come il software da cambiare interagisce con essi, il che è molto difficile.

Manutenzione ed evoluzione del software – Costi

- Qualità della formattazione e dello stile del codice: i pasticci creano sempre necessità di manutenzione.
  - Forse il problema peggiore è quando i nomi sono scelti male.
- Età e struttura del programma: man mano che i programmi invecchiano, i cambiamenti cominciano a degradare la loro struttura, rendendoli più difficili da capire e cambiare.

- Alcuni tipici fattori di costo umano sono i seguenti:
  - Stabilità del team: i costi di manutenzione sono maggiori se gli sviluppatori originali non sono disponibili.
  - Responsabilità contrattuale: se non si prevede che gli sviluppatori originali siano responsabili dei cambiamenti futuri, questi si sentono poco incentivati a progettare per il cambiamento.
  - Competenze del personale: i compiti di manutenzione sono a volte affidati a ingegneri principianti, stagisti estivi, o programmatori poco istruiti e poco qualificati.
  - Questo significa che possono richiedere più tempo e i loro cambiamenti possono essere meno affidabili.

Manutenzione ed evoluzione del software – Terminologia

- Vediamo alcune definizioni che vengono utilizzate nel campo della manutenzione ed evoluzione del software:
  - Manutenibilità:
    - La facilità con cui un sistema o un componente software può essere modificato per correggere i difetti, migliorare le prestazioni o altri attributi, o adattarsi a un ambiente cambiato.
    - La manutenibilità non può essere misurata direttamente, ma è influenzata da fattori come l'età, la dimensione, il linguaggio di programmazione, l'architettura e il design, la documentazione e la formattazione.
    - La manutenibilità è anche abbastanza soggettiva, poiché la capacità dell'ingegnere di capire il software è pesantemente influenzata dalla conoscenza ed esperienza precedente con altri programmi.
  - Effetto a catena (Ripple Effect):
    - I cambiamenti in una posizione del software possono avere un impatto su altri componenti.
    - Gli effetti Ripple non possono essere pienamente compresi con l'analisi statica del sorgente, quindi si deve usare l'analisi dinamica. Cioè, dobbiamo eseguire il programma.

Manutenzione ed evoluzione del software – Terminologia

- Analisi dell'impatto:
  - Il processo di identificazione delle potenziali conseguenze di un cambiamento in termini di come esso influenzerà il resto del sistema.
  - Questa è un'analisi difficile che viene fatta per stimare il costo di un cambiamento, per scegliere quale riparazione del guasto è più conveniente, per pianificare i test di regressione, o per capire quali risorse saranno necessarie per effettuare il cambiamento.
- Tracciabilità:
  - Il grado in cui si può stabilire una relazione tra due o più prodotti del processo di sviluppo, come i requisiti e il codice, o la documentazione di progettazione e i test.
- Sistemi legacy:
  - Un sistema software che è ancora in uso ma il team di sviluppo non è più disponibile.
  - I sistemi legacy sono stati spesso ereditati, hanno ancora valore e sono molto difficili da cambiare.

Manutenzione ed evoluzione del software – Reality Check

- Come detto sopra, la maggior parte della letteratura disponibile sulla manutenzione e la regressione è vecchia di 30 anni!
- Gli standard IEEE sono del 1990, e sono cambiati solo modestamente dalle loro versioni del 1983.
- I principali libri di testo sono stati aggiornati regolarmente dagli anni '80, le informazioni sulla manutenzione non sono cambiate molto.
- La letteratura è anche conflittuale sull'uso di manutenzione ed evoluzione.
- Una possibile distinzione è:
  - Manutenzione del software: le attività richieste per mantenere un sistema software operativo e reattivo dopo che è stato distribuito.
  - Evoluzione del software: Un cambiamento continuo da uno stato inferiore, più semplice o peggiore ad uno stato superiore o migliore.
- La manutenzione del software riguarda la correzione del software in modo che sia vicino a funzionare come previsto (adattiva, correttiva e preventiva), mentre l'evoluzione riguarda il cambiamento del comportamento previsto (perfettiva).

Manutenzione ed evoluzione del software – Leggi di Lehman sulla evoluzione del software

- Prima di concludere, qualsiasi panoramica sulla manutenzione o l'evoluzione è necessario dare un cenno alle leggi di Lehman.
- Queste sono state proposte per la prima volta intorno al 1980, e sembrano essere rilevanti ancora oggi.
- Esse sono le seguenti:
  - Legge del cambiamento continuo: il software che è usato in un ambiente del mondo reale deve cambiare o diventerà sempre meno utile in quell'ambiente.
  - Legge della complessità crescente: man mano che un programma in evoluzione cambia, la sua struttura diventa più complessa, a meno che non si facciano sforzi attivi per evitare questo fenomeno (quindi refactoring).
  - Legge dell'autoregolazione: l'evoluzione del programma è un processo di autoregolazione; gli attributi del sistema come la dimensione, il tempo tra i rilasci e il numero di errori riportati sono approssimativamente invarianti per ogni rilascio del sistema.
  - Legge di conservazione della stabilità organizzativa: durante la vita di un programma, il suo tasso di sviluppo è approssimativamente costante e indipendente dalle risorse dedicate allo sviluppo del sistema.

Manutenzione ed evoluzione del software – Leggi di Lehman sulla evoluzione del software

- Legge di conservazione della familiarità: nel corso della vita di un sistema, il cambiamento incrementale del sistema in ogni release è approssimativamente costante.
- Legge della crescita continua: le funzionalità offerte dai sistemi devono aumentare continuamente per mantenere la soddisfazione dell'utente.
- Legge della qualità decrescente: la qualità di un sistema apparirà in declino a meno che non si adatti ai cambiamenti nel suo ambiente operativo.
- Legge del sistema di feedback: i processi di evoluzione incorporano sistemi di feedback multi-agente e multi-loop e bisogna trattarli come sistemi di feedback per ottenere un miglioramento significativo del prodotto.

Configuration Management

- Durante gli anni '50 il Dipartimento della Difesa degli Stati Uniti sviluppò una disciplina di gestione tecnica per seguire i cambiamenti nello sviluppo di sistemi complessi.
- Esso diede a questo sistema e a varie iterazioni nomi molto tecnici, finché nel 2001 pubblicò una guida consolidata che definì le caratteristiche di quel sistema di gestione tecnica ora chiamato gestione della configurazione.
- Oggi, la gestione della configurazione non è usata solo dal dipartimento della difesa, ma nello sviluppo del software, nella gestione dei servizi IT, nell'ingegneria civile, nell'ingegneria industriale e altro ancora.
- La gestione della configurazione è un processo di ingegneria dei sistemi per stabilire la coerenza degli attributi di un prodotto durante la sua vita.
- Nel mondo della tecnologia, la gestione della configurazione è un processo di gestione IT che tiene traccia dei singoli elementi di configurazione di un sistema IT.
- I sistemi IT sono composti da asset IT che variano in granularità.
- Un asset IT può rappresentare un pezzo di software, o un server, o un cluster di server.

Configuration Management

- Nel seguito ci concentreremo sulla gestione della configurazione per come si applica direttamente alle risorse software IT e ai processi di Continuous Integration / Continuous Delivery delle risorse software.

- La gestione della configurazione del software è un processo di ingegneria dei sistemi che traccia e monitora le modifiche ai metadati di configurazione di un sistema software.

- Nello sviluppo del software, la gestione della configurazione è comunemente usata insieme al controllo della versione e all'infrastruttura CI/CD.

Configuration Management - Importanza

- La gestione della configurazione aiuta i team di ingegneri a costruire sistemi robusti e stabili attraverso l'uso di strumenti che gestiscono e monitorano automaticamente gli aggiornamenti dei dati di configurazione.
- I sistemi software complessi sono costituiti da componenti che differiscono per granularità di dimensioni e complessità.
- Per un esempio più concreto si consideri un'architettura a microservizi.
- Ogni servizio in un'architettura a microservizi usa metadati di configurazione per registrarsi e inizializzarsi.
- Alcuni esempi di metadati di configurazione del software sono:
  - Specifiche di allocazioni di risorse hardware computazionali per CPU, RAM, etc.
  - Endpoint che specificano le connessioni esterne ad altri servizi, database o domini.
  - Segreti come password e chiavi di crittografia.
- È facile che questi valori di configurazione vengano considerati come qualcosa la cui gestione si può rinviare, portando la configurazione a diventare disorganizzata e sparpagliata.

Configuration Management - Importanza

- Immaginiamoci numerosi post-it con password e URL in giro per un ufficio.
- La gestione della configurazione risolve questo problema creando una "fonte di verità" con una locazione centrale per la configurazione.
- Git è una piattaforma fantastica per gestire i dati di configurazione.
- Spostare i dati di configurazione in un repository Git permette il controllo della versione e consente al repository di agire come fonte di verità.
- Il controllo di versione risolve anche un altro problema di configurazione, ovvero quello delle modifiche inaspettate.
- Gestire i cambiamenti inaspettati attraverso l'uso della revisione del codice e del controllo di versione aiuta a minimizzare i tempi di inattività.
- I valori di configurazione saranno spesso aggiunti, rimossi o modificati.
- Senza il controllo di versione questo può causare problemi.

Configuration Management - Importanza

- Un membro del team può modificare un valore di allocazione dell'hardware in modo che il software funzioni in modo più efficiente sul suo portatile personale.
- Quando il software viene poi distribuito in un ambiente di produzione, questa nuova configurazione può avere un effetto subottimale o può non funzionare.
- Il controllo della versione e la gestione della configurazione risolvono questo problema aggiungendo visibilità alle modifiche della configurazione.
- Quando viene fatta una modifica ai dati di configurazione, il sistema di controllo della versione la traccia, il che permette ai membri del team di rivedere un audit trail delle modifiche.
- Il controllo della versione della configurazione permette il rollback o la funzionalità di "annullamento" della configurazione, che aiuta ad evitare rotture inaspettate.
- Il controllo di versione applicato alla configurazione può essere rapidamente riportato all'ultimo stato stabile conosciuto.

Configuration Management – Come avviene

- La gestione della configurazione consiste in una serie di attività che vengono schematizzati nella seguente figura:



- Essi sono:
  - Identificazione:
    - La prima azione verso la gestione della configurazione è la raccolta di informazioni.
    - I dati di configurazione dovrebbero essere aggregati e compilati da diversi ambienti applicativi, sviluppo, staging e produzione per tutti i componenti e servizi in uso.
    - Tutti i dati segreti come le password e le chiavi dovrebbero essere identificati e crittografati e archiviati in modo sicuro.
    - A questo punto i dati di configurazione dovrebbero essere organizzati in file di dati che possono essere indicati come una fonte centrale di verità.

Configuration Management – Come avviene

- Baseline:
  - Dopo che i dati di configurazione sono stati aggregati e organizzati, si può stabilire una baseline.
  - Una configurazione baseline è uno stato noto di configurazione che farà funzionare con successo il software dipendente senza errori.
  - Questa baseline viene solitamente creata rivedendo la configurazione di un ambiente di produzione funzionante e registrando quelle impostazioni di configurazione.
- Controllo di versione:
  - Il progetto di sviluppo dovrebbe usare un sistema di controllo delle versioni.
  - È importante assicurarsi che tutti i dati segreti come password o chiavi siano criptati con una chiave esterna.
  - I dati segreti registrati accidentalmente in un repository sono un rischio enorme.
  - Devono essere cancellati dalla storia dei repository o saranno a rischio di essere utilizzati.

Configuration Management – Come avviene

- Auditing:
  - Avere dati di configurazione organizzati e aggiunti a un repository permette la collaborazione e la visibilità della configurazione del sistema.
  - Il popolare pull request workflow che i team di software usano per rivedere e modificare il codice può essere applicato ai file di dati di configurazione.
  - Questo aiuta a costruire un sistema di controllo e di contabilità.
  - Qualsiasi modifica applicata alla configurazione deve essere rivista e accettata dal team.
  - Questo aggiunge responsabilità e visibilità nei cambiamenti di configurazione.

Configuration Management – Rapporto con le metodologie agili

- I dati di configurazione sono stati storicamente difficili da gestire e possono facilmente diventare un problema continuamente rinviato.
- Non è vero codice, quindi non viene immediatamente messo nel controllo di versione, e non è un dato di prima classe, quindi non viene memorizzato in un database primario.
- L'amministrazione di sistema tradizionale e su piccola scala è di solito fatta con un insieme di script e processi ad-hoc.
- I dati di configurazione possono essere a volte trascurati, ma sono fondamentali per il funzionamento del sistema.
- L'ascesa delle infrastrutture cloud ha portato allo sviluppo e all'adozione di nuovi modelli di gestione dell'infrastruttura.
- Le complesse architetture di sistema basate sul cloud sono gestite e distribuite attraverso l'uso di file di dati di configurazione.
- Queste nuove piattaforme cloud permettono ai team di specificare le risorse hardware e le connessioni di rete di cui hanno bisogno attraverso file di dati leggibili dall'uomo e dalla macchina come YAML. I file di dati vengono quindi letti e l'infrastruttura viene fornita nel cloud. Questo modello è chiamato Infrastructure as Code (IaC).

Configuration Management – Rapporto con le metodologie agili

- La gestione della configurazione permette ai team agili di classificare chiaramente e dare priorità al lavoro di configurazione.
- Esempi di lavoro di configurazione sono attività e task quali:
  - Aggiornare i certificati SSL di produzione.
  - Aggiungere un nuovo endpoint del database.
  - Cambiare la password per i servizi email di dev, staging e produzione..
  - Aggiungere chiavi API per una nuova integrazione di terze parti.
- Una volta che una piattaforma di gestione della configurazione è presente, i team hanno visibilità sul lavoro richiesto per i task di configurazione.
- Il lavoro di gestione della configurazione può essere identificato come dipendenze per altri lavori e affrontato correttamente come parte degli sprint agili.

Configuration Management – Tool - Git

- Git è il sistema di controllo di versione leader del settore per tracciare le modifiche al codice.
- L'aggiunta di dati di gestione della configurazione insieme al codice in un repository Git fornisce una visione olistica del controllo di versione di un intero progetto.
- Git è uno strumento fondamentale nella gestione della configurazione di livello superiore.

Configuration Management – Tool - Docker

- Docker ha introdotto la containerizzazione che è una forma avanzata di gestione della configurazione.
- Docker si basa su file di configurazione chiamati Dockerfiles, che contengono una lista di comandi che vengono valutati per ricostruire la snapshot prevista dello stato del sistema operativo.
- Docker crea contenitori da questi Dockerfile che sono istantanee di un'applicazione preconfigurata.
- I Dockerfile sono memorizzati in un repository Git per il monitoraggio delle versioni e hanno bisogno di un'ulteriore gestione della configurazione per il loro deployment nell'infrastruttura.

Configuration Management – Tool - Terraform

- Terraform è una piattaforma open source di gestione della configurazione fornita da HasiCorp.
- Terraform utilizza IaC per fornire e gestire cluster, infrastrutture cloud o servizi.
- Terraform supporta Amazon Web Services (AWS), Microsoft Azure e altre piattaforme cloud.
- Ogni piattaforma cloud ha la propria rappresentazione e interfaccia per i componenti comuni dell'infrastruttura come server, database e code.
- Terraform ha costruito un livello di astrazione di strumenti di configurazione per le piattaforme cloud che permettono ai team di scrivere file che sono definizioni riproducibili della loro infrastruttura.

Configuration Management – Tool – Altri tool

- Ansible, Salt Stack e Chef sono framework di automazione IT.
- Questi framework automatizzano molti processi tradizionali degli amministratori di sistema.
- Ogni framework usa una serie di file di dati di configurazione -- di solito YAML o XML -- che sono valutati da un eseguibile.
- I file di dati di configurazione specificano una sequenza di azioni da intraprendere per configurare un sistema.
- Le azioni sono poi eseguite dall'eseguibile.
- L'eseguibile differisce nel linguaggio tra i sistemi -- Ansible e Salt Stack sono basati su Python e Chef è basato su Ruby.
- Questo flusso di lavoro è simile all'esecuzione di script di shell ad-hoc, ma offre un'esperienza più strutturata e raffinata attraverso gli ecosistemi delle rispettive piattaforme.
- Questi strumenti sono ciò che porteranno all'automazione necessaria per raggiungere il CI/CD.

Configuration Management – In conclusione

- La gestione della configurazione è uno strumento necessario per gestire sistemi software complessi.
- La mancanza di gestione della configurazione può causare seri problemi di affidabilità, perdita di tempo e capacità di scalare un sistema.
- Molti strumenti di sviluppo software attuali hanno funzioni di gestione della configurazione incorporate.
- Bitbucket offre un potente sistema per la gestione della configurazione che è costruito intorno ai flussi di lavoro delle richieste di pull di Git e alle pipeline CI/CD.

Qualità del software - Introduzione

- La qualità della soluzione software può essere intesa sia in termini di proprietà intrinseche del software, sia in termini di ciò che è percepito dal committente.

- Per chiarire meglio questa distinzione, si può pensare che la qualità intesa in termini di proprietà intrinseche del software riguarda tutti gli aspetti che rispondono alla domanda "il programma è giusto?", mentre la qualità percepita dal committente è relativa a tutti gli aspetti che rispondono alla domanda "è il giusto programma?".

- Tipicamente le domande considerate per valutare se il programma "è giusto" sono le seguenti:

  - Il software è affidabile?

  - Quanto è efficiente?

  - Quanto è semplice modificarlo?

  - Quanto costa trasferire il software da un ambiente a un altro?

Qualità del software - Introduzione

- Per valutare invece se è “il giusto programma”, ci si pone le seguenti domande:
  - Le funzionalità offerte dal software soddisfano i fabbisogni del committente?
  - Il committente riconosce e comprende le funzionalità del software?
- In generale, la qualità di un prodotto software dipende non solo dalla qualità della soluzione software – come erroneamente si potrebbe essere portati a credere – ma anche dalla qualità di tutti gli altri elementi di cui si compone il prodotto.
- Per esempio, a molti è sicuramente capitato di utilizzare applicazioni software ottime dal punto di vista funzionale, per le prestazioni e la ricchezza delle funzionalità offerte, ma prive di un esaustivo manuale utente o help online.
- In questo caso l’assenza di qualità della manualistica disponibile ha un effetto negativo sulla qualità totale del software: l’utente come può infatti percepire e quindi apprezzare appieno le funzionalità di un’applicazione software se non viene messo in condizione di apprendere come possono essere utilizzate?
- Si ottiene in questo caso una percezione di qualità complessiva piuttosto bassa: quanto maggiore è la ricchezza delle funzionalità e scadente la documentazione di corredo, tanto maggiore sarà la percezione dell’utente di essere di fronte a un’applicazione estremamente complessa, poco usabile e, in generale, non soddisfacente.

Qualità del software - Introduzione

- La qualità del software può essere definita secondo tre differenti viste: qualità interna, qualità esterna e qualità in uso.
  - La qualità interna si riferisce alle caratteristiche interne del prodotto software, cioè a tutte quelle proprietà che sono rilevanti durante le fasi di sviluppo e di manutenzione del software stesso.
  - La qualità esterna descrive invece tutte le proprietà del software che lo caratterizzano durante la sua esecuzione.
  - La qualità in uso, infine, descrive le proprietà del prodotto software in uno specifico ambiente di esecuzione o contesto d'uso (quindi dal punto di vista di chi utilizza il software).
- Tali viste di qualità sono applicabili a qualsiasi tipologia di software.
- Per descrivere le tre viste di qualità, è possibile considerare due modelli, uno per le qualità interna ed esterna e uno per quella in uso.
- Entrambi i modelli definiscono la qualità in termini di un insieme di caratteristiche organizzate secondo una specifica struttura.

Qualità del software - Introduzione

- In particolare, il modello per la qualità interna ed esterna del software fornisce un insieme di 6 caratteristiche e di 27 sottocaratteristiche, mentre il modello per la qualità in uso definisce altre 4 caratteristiche.

- La seguente tabella illustra sinteticamente i due modelli e le rispettive caratteristiche/sottocaratteristiche.

Modello	Caratteristica	Sottocaratteristica
Qualità interna ed esterna	Funzionalità	Adeguatezza Accuratezza Interoperabilità Aderenza standard (Conformità) Sicurezza
	Usabilità	Comprensibilità Apprendibilità Operabilità Attrattività Conformità
	Affidabilità	Maturità Tolleranza ai guasti Ripristinabilità Conformità
	Efficienza	Comportamento temporale Consumo di risorse Conformità
	Manutenibilità	Analizzabilità Modificabilità Stabilità Testabilità Conformità
	Portabilità	Adattabilità Installabilità Coesistenza Sostituibilità Conformità
Qualità in uso	Efficacia Produttività Sicurezza Soddisfazione	

Qualità del software – Il modello di qualità interna ed esterna

- Questo modello stabilisce che la qualità del software dipende dalla presenza delle seguenti sei caratteristiche di qualità:
  - funzionalità,
  - usabilità,
  - affidabilità,
  - efficienza,
  - manutenibilità,
  - portabilità.
- Per ciascuna di esse viene definito un insieme di sottocaratteristiche che meglio descrivono la caratteristica di qualità.

Qualità del software – Il modello di qualità interna ed esterna – Funzionalità (functionality)

- Per funzionalità di un prodotto software si intende la sua capacità di svolgere compiti che abbiano una qualche utilità in relazione a bisogni espressi o impliciti degli utenti.

- Per esempio, nel caso di un word processor le funzionalità sono quelle relative alla capacità di formattare il testo, di impaginarlo, di farne il controllo ortografico, di applicarci dei template o di crearne l'anteprima di stampa.

- Per meglio definire che cosa si intende per funzionalità del software, lo standard indica cinque sottocaratteristiche: adeguatezza, accuratezza, interoperabilità, aderenza agli standard e sicurezza.

  - Adeguatezza (suitability):

    - Per adeguatezza del prodotto software si intende la sua capacità di svolgere un insieme di funzioni che risultino appropriate per un compito o un obiettivo specifico.

    - Si consideri per esempio un'applicazione per il calcolo delle tasse e si ipotizzi che essa recepisca le regole fiscali vigenti in Italia.

      - Tale applicazione risulterà adeguata se utilizzata per calcolare le tasse da pagare in Italia, ma del tutto inadeguata se utilizzata per calcolare le tasse in paesi che adottano regole fiscali diverse da quelle italiane.

Qualità del software – Il modello di qualità interna ed esterna – Funzionalità (functionality)

- Accuratezza (accuracy):
  - L'accuratezza esprime la capacità del prodotto software di fornire i risultati e gli effetti corretti e attesi.
  - Per esempio, il software che calcola le tasse da versare allo stato deve essere accurato in quanto deve calcolare con precisione assoluta l'importo dovuto.
  - In altri casi, l'accuratezza può essere espressa con il margine di errore tollerato nel risultato fornito.
- Interoperabilità (interoperability):
  - L'interoperabilità riguarda la capacità del prodotto software di interagire con altri sistemi.
  - Nel mondo dei sistemi informativi, un esempio di interoperabilità è dato dalla possibilità di interconnettere diverse applicazioni attraverso il pattern pipe and filter, ridirigendo gli input e gli output.
  - Gli esempi di tecnologie più moderne che hanno lo scopo di garantire e favorire lo sviluppo di software interoperabile sono XML e JSON.

Qualità del software – Il modello di qualità interna ed esterna – Funzionalità (functionality)

- Aderenza agli standard (compliance):
  - L'aderenza agli standard si riferisce alla capacità del software di rispettare protocolli, leggi, convenzioni, prescrizioni e standard.
  - Per esempio, vi sono applicazioni Java sviluppate secondo il modello architetturale J2EE (Java 2 Enterprise Edition): tale modello indica quali siano gli standard che devono essere adottati dalle applicazioni web-based, dalle applicazioni remote, dalle applicazioni che accedono a dati mantenuti in database ecc.
  - È importante evidenziare il forte nesso tra l'aderenza agli standard e l'interoperabilità: molti degli standard esistenti, in particolare quelli relativi alla comunicazione, sono proprio finalizzati a facilitare l'interoperabilità.
- Sicurezza (security):
  - La sicurezza del prodotto software esprime la sua capacità di prevenire accessi non autorizzati e di resistere ad attacchi intenzionali mirati ad accedere a informazioni confidenziali, a modificare dati o il software stesso, a fornire qualche vantaggio a chi svolge gli attacchi e a impedire che i legittimi utenti possano utilizzare il prodotto software.

Qualità del software – Il modello di qualità interna ed esterna – Usabilità (usability)

- Un prodotto software si dice usabile se viene compreso e appreso dall'utente senza difficoltà e viene quindi utilizzato in modo naturale e immediato.

- Tale caratteristica risulta particolarmente critica perché spesso il successo di un prodotto software dipende più dalla percezione che ne ha l'utente che da altre caratteristiche quali ad esempio la ricchezza delle funzionalità.

- Accade spesso, infatti, che gli utenti preferiscano un prodotto software che offre poche funzionalità che risultano semplici e intuitive da utilizzare, piuttosto che applicazioni software con tante funzionalità la cui complessità d'uso le rende di fatto inutilizzabili.

- L'usabilità viene scomposta in cinque sottocaratteristiche: comprensibilità, apprendibilità, operabilità, attrattività e conformità.

  - Comprensibilità (understandability):

    - Il software si dice comprensibile quando un utente è in grado di capire se è adatto per le proprie esigenze e se è in grado di utilizzarlo per svolgere determinati compiti.

Qualità del software – Il modello di qualità interna ed esterna – Usabilità (usability)

- Tutte le applicazioni software che si adeguano alle linee guida dei sistemi grafici a finestre per l'interfaccia utente (ad esempio quelli di Windows, Mac, OS/2 ecc.) sono un ottimo esempio di applicazione software comprensibile: infatti in tutte le applicazioni le funzionalità di gestione del file si trovano in un menù a tendina che occupa sempre la stessa posizione; per chiudere il programma è messa a disposizione sempre la stessa icona; in tutte le applicazioni è sempre presente un'icona che attiva il salvataggio del file ecc.

- Apprendibilità (learnability):

  - Il livello di apprendibilità di un prodotto software indica lo sforzo necessario perché gli utenti ne apprendano il corretto utilizzo.

  - Esistono applicazioni software la cui modalità d'utilizzo risulta particolarmente intuitiva grazie al fatto che le funzionalità offerte sono alquanto semplici.

  - Ciò non significa che l'applicazione sia apprendibile: l'intuitività non va infatti confusa con l'apprendibilità, che serve proprio a comprendere quanto sia facile imparare l'uso di funzionalità che non siano di intuitiva comprensione.

Qualità del software – Il modello di qualità interna ed esterna – Usabilità (usability)

- Operabilità (operability):
  - L'operabilità del prodotto software indica lo sforzo necessario agli utenti per eseguirne e controllarne le varie operazioni.
  - Confrontando tra di loro due applicazioni software che offrono le stesse funzionalità (ad esempio due differenti fogli di calcolo), risulta più operabile l'applicazione che consente di svolgere le operazioni con il minor sforzo per l'utente (minor numero di click del mouse, minor numero di pressioni di tasti ecc.).
- Attrattività (attractiveness):
  - L'attrattività esprime la capacità di un prodotto software di risultare piacevole all'uso per l'utente.
- Conformità (compliance):
  - Un prodotto software risulta conforme per quanto concerne l'usabilità se rispetta standard, linee guida, convenzioni o regole di usabilità.
  - Un esempio classico è fornito dai portali sviluppati per la Pubblica Amministrazione: perché siano conformi (e quindi approvati dal committente) devono essere realizzati rispettando gli standard di navigabilità per le persone ipovedenti.

Qualità del software – Il modello di qualità interna ed esterna – Affidabilità (reliability)

- L'affidabilità di un prodotto software esprime la sua capacità di mantenere un livello adeguato di prestazioni sotto determinate condizioni e per un periodo di tempo fissato.

- Si dettaglia nelle seguenti sottocaratteristiche di qualità: maturità, tolleranza ai guasti, ripristinabilità e conformità.

  - Maturità (maturity):

    - Un prodotto software si dice maturo se la presenza di guasti (condizione di malfunzionamento anomalo interno al programma) non si manifesta come malfunzionamento (manifestazione di un guasto visibile dall'utente) del prodotto stesso.

    - Quindi, più un prodotto è maturo, meno frequentemente l'utente riscontra dei malfunzionamenti.

  - Tolleranza ai guasti (fault tolerance):

    - La tolleranza ai guasti esprime la capacità del prodotto software di mantenere un determinato livello di prestazioni in presenza di guasti nel software o dell'uso scorretto delle interfacce.

    - Come esempio si pensi a quei programmi che sono in grado, anziché interrompersi bruscamente, di rilevare un proprio malfunzionamento e di continuare a operare anche se con funzionalità ridotte.

Qualità del software – Il modello di qualità interna ed esterna – Affidabilità (reliability)

- Ripristinabilità (recoverability):
  - La ripristinabilità indica la capacità del prodotto software di ristabilire il livello delle prestazioni e di recuperare i dati interessanti antecedenti a un malfunzionamento, nonché il tempo e lo sforzo necessario per tali operazioni.
  - Un esempio di applicazione ripristinabile è relativo a quegli applicativi che offrono la funzionalità di autoriparazione.
  - Tale funzionalità permette infatti di controllare se i file di programma, le librerie e le informazioni di configurazione non si siano corrotti nel tempo e, se necessario, è in grado di ripristinare i file originali.
- Conformità (compliance):
  - Un prodotto software risulta conforme per quanto concerne l'affidabilità se rispetta standard, convenzioni o regole di affidabilità.

Qualità del software – Il modello di qualità interna ed esterna – Efficienza (efficiency)

- L'efficienza misura il rapporto tra il livello delle prestazioni del prodotto e la quantità di risorse che devono essere impiegate per raggiungere tali prestazioni.

- Per valutare l'efficienza del software è quindi necessario analizzare da un lato il comportamento temporale e dall'altro il consumo di risorse.

  - Comportamento temporale (time behavior):

    - Il comportamento temporale assunto dal software si riferisce ai tempi di risposta e di elaborazione, nonché al tempo necessario per smaltire le richieste di esecuzione delle sue funzionalità.

  - Consumo di risorse (resource utilization):

    - Il consumo di risorse indica la quantità, il tipo di risorse utilizzate e la durata del loro impiego necessari per l'esecuzione delle funzionalità di un prodotto software.

  - Conformità (compliance):

    - Un prodotto software risulta conforme per quanto concerne l'efficienza se rispetta standard e convenzioni relative all'efficienza.

Qualità del software – Il modello di qualità interna ed esterna – Manutenibilità (maintainability)

- La manutenibilità del prodotto software indica la capacità del software di subire modifiche come correzioni, miglioramenti o adattamenti a seguito di cambiamenti dell'ambiente, dei requisiti o delle specifiche funzionali.

- La manutenibilità può essere scomposta nelle seguenti sottocaratteristiche: analizzabilità, modificabilità, stabilità, testabilità e conformità.

  - Analizzabilità (analyzability):

    - Un prodotto software si dice analizzabile se consente da un lato di diagnosticare le inadeguatezze e le cause dei malfunzionamenti e, dall'altro, di identificare le porzioni di prodotto su cui è necessario intervenire per apportare una modifica.

  - Modificabilità (changeability):

    - Il livello di modificabilità è un'indicazione dello sforzo necessario per apportare una determinata modifica al prodotto software.

  - Stabilità (stability):

    - Un prodotto software si dice stabile se, apportandovi delle modifiche, non si riscontrano o sono ridotti al minimo gli effetti inattesi dovuti alle stesse.

Qualità del software – Il modello di qualità interna ed esterna – Manutenibilità (maintainability)

- Testabilità (testability):
  - La testabilità del prodotto software esprime lo sforzo necessario per validare il prodotto dopo le modifiche.
- Conformità (compliance)
  - Un prodotto software risulta conforme per quanto concerne la manutenibilità se rispetta standard e convenzioni di manutenibilità.

Qualità del software – Il modello di qualità interna ed esterna – Portabilità (portability)

- Un prodotto software si dice portabile quando può essere facilmente trasferito da un ambiente a un altro (per ambiente si può intendere, per esempio, il sistema operativo, il software di sistema, il middleware, l'ambiente grafico).

- Per comprendere quanto un prodotto software sia portabile occorre valutare in che misura il software risulta essere adattabile, installabile, sostituibile, conforme e fino a che punto sia in grado di convivere con altre applicazioni.

- Tali proprietà corrispondono infatti alle sottocaratteristiche di qualità nelle quali è possibile suddividere la portabilità.

  - Adattabilità (adaptability):

    - L'adattabilità di un prodotto software esprime la capacità di adattarsi ad ambienti diversi da quello di origine senza che sia necessario applicare azioni o strumenti oltre a quelli previsti e messi appositamente a disposizione dal software stesso.

    - Un esempio classico di adattabilità è dato dalle applicazioni sviluppate in Java e più in generale, dalle applicazioni sviluppate su una virtual machine disponibile su ambienti operativi diversi.

Qualità del software – Il modello di qualità interna ed esterna – Portabilità (portability)

- Installabilità (installability):
  - L'installabilità indica lo sforzo necessario per installare il software in uno specifico ambiente diverso da quello di origine.
  - È ormai irrinunciabile per un prodotto software la disponibilità di un wizard (o di meccanismi analoghi) che, attraverso una procedura guidata, determina tutti i parametri di installazione e personalizzazione del prodotto, ed evita all'utente di dover effettuare manualmente molte operazioni.
- Coesistenza (co-existence)
  - La coesistenza si riferisce a quanto un prodotto software sia in grado di coesistere in uno stesso ambiente con altre applicazioni indipendenti e di condividere con esse delle risorse comuni.
  - Un esempio in cui tale qualità è assente è fornito da quelle applicazioni che smettono inspiegabilmente di funzionare correttamente a seguito dell'installazione di un altro pacchetto apparentemente totalmente scorrelato.
  - Per quanto riguarda invece la condivisione delle risorse comuni, tutti i moderni sistemi operativi danno la possibilità alle applicazioni software di condividere ad esempio le librerie dinamiche (consentendo quindi di poterle installare una volta sola).

Qualità del software – Il modello di qualità interna ed esterna – Portabilità (portability)

- Sostituibilità (replaceability):
  - Un prodotto software si dice sostituibile quando è possibile utilizzarlo al posto di un altro specifico prodotto che abbia lo stesso scopo e si riferisca allo stesso ambiente.
  - Un esempio tipico è dato dal proliferare delle versioni open source di pacchetti di office automation che consentono di sostituire quasi completamente i corrispettivi pacchetti commerciali.
- Conformità (compliance):
  - Un prodotto software risulta conforme per quanto concerne la portabilità se rispetta standard e convenzioni relative alla portabilità.

Qualità del software – Il modello di qualità in uso

- Il modello di qualità in uso descrive le caratteristiche di qualità di un prodotto software dal punto di vista dell'utente che lo utilizza in uno specifico contesto.
- In particolare, esso ne definisce quattro: l'efficacia, la produttività, la sicurezza e la soddisfazione.
- Secondo lo standard, ciascuna di esse va valutata per uno specifico contesto d'uso.
  - Efficacia (effectiveness):
    - Un prodotto software è efficace se permette a un utente di raggiungere i propri obiettivi completamente e con accuratezza.
    - Questa proprietà è strettamente legata al fatto che i bisogni dell'utente siano soddisfatti, cioè che l'utente ritrovi tra le funzionalità offerte dal prodotto quelle che gli permettono di svolgere il compito che si è prefissato.
  - Produttività (productivity):
    - Un prodotto software si dice produttivo se gli utenti, per utilizzare efficacemente il prodotto, devono spendere una limitata quantità di risorse (economiche, tempo di utilizzo, potenza di calcolo, memoria ecc.).

Qualità del software – Il modello di qualità in uso

- Sicurezza (safety):
  - La sicurezza si riferisce alla capacità del prodotto software di garantire accettabili livelli di rischio di danni alle persone, al software, al business, alle proprietà e all'ambiente.
  - Senza ricorrere ad esempi estremi quali i noti fallimenti in ambito aerospaziale, basta pensare a tutte le applicazioni che governano le transazioni bancarie, piuttosto che al firmware che governa gli elettrodomestici più evoluti (ad esempio il firmware di una lavatrice deve mantenere bloccata la porta dell'oblò finché il cestello non si è completamento svuotato d'acqua).
- Soddisfazione (satisfaction):
  - La soddisfazione indica la capacità del prodotto software di soddisfare gli utenti.
  - Una misura indiretta della soddisfazione di un utente è data ad esempio dall'assiduità con cui l'utente utilizza il prodotto.

Riuso del software - Introduzione

- Nella maggior parte delle discipline ingegneristiche, i sistemi sono progettati mettendo insieme componenti esistenti che sono stati usati in altri sistemi.
- L'ingegneria del software è stata più focalizzata sullo sviluppo originale, ma è ora riconosciuto che per ottenere un software migliore, più rapidamente e a costi inferiori, abbiamo bisogno di un processo di progettazione che si basa sul riutilizzo sistematico del software.
- C'è stato un importante passaggio allo sviluppo basato sul riuso negli ultimi 10 anni.
- Esistono vari livelli di riuso del software:
  - Riuso dei sistemi: vengono riutilizzati sistemi completi, che possono includere diversi programmi applicativi.
  - Riuso delle applicazioni: un'applicazione può essere riutilizzata incorporandola senza modifiche in altre o sviluppando famiglie di applicazioni.
  - Riuso dei componenti: i componenti di un'applicazione, dai sottosistemi ai singoli oggetti, possono essere riutilizzati.
  - Riuso di oggetti e funzioni: componenti software su piccola scala che implementano un singolo oggetto o funzione ben definita possono essere riutilizzati.

Riuso del software - Introduzione

- Il riuso del software presenta i seguenti vantaggi:
  - Sviluppo accelerato:
    - Portare un sistema sul mercato il prima possibile è spesso più importante dei costi complessivi di sviluppo.
    - Riutilizzare il software può accelerare la produzione del sistema perché sia il tempo di sviluppo che quello di validazione possono essere ridotti.
  - Uso efficace degli specialisti:
    - Invece di fare lo stesso lavoro più e più volte, gli specialisti delle applicazioni possono sviluppare software riutilizzabile che incapsula le loro conoscenze.
  - Maggiore affidabilità:
    - Il software riutilizzato, che è stato provato e testato in sistemi funzionanti, dovrebbe essere più affidabile del software nuovo.
    - I suoi difetti di progettazione e implementazione dovrebbero essere stati trovati e corretti.

Riuso del software - Introduzione

- Minori costi di sviluppo:
  - I costi di sviluppo sono proporzionali alla dimensione del software da sviluppare.
  - Riutilizzare il software significa che devono essere scritte meno righe di codice.
- Rischio di processo ridotto:
  - Il costo del software esistente è già noto, mentre i costi di sviluppo sono sempre una questione di giudizio.
  - Questo è un fattore importante per la gestione del progetto perché riduce il margine di errore nella stima dei costi del progetto.
  - Questo è particolarmente vero quando vengono riutilizzati componenti software relativamente grandi come i sottosistemi.

Riuso del software - Introduzione

- Conformità agli standard:
  - Alcuni standard, come quelli dell'interfaccia utente, possono essere implementati come un insieme di componenti riutilizzabili.
  - Per esempio, se i menu in un'interfaccia utente sono implementati usando componenti riutilizzabili, tutte le applicazioni presentano agli utenti gli stessi formati di menu.
  - L'uso di interfacce utente standard migliora l'affidabilità perché gli utenti fanno meno errori quando vengono presentati con un'interfaccia familiare.

- Il riuso del software presenta però, anche, i seguenti problemi:
  - Creare, mantenere e usare una libreria di componenti:
    - Popolare una libreria di componenti riutilizzabili e assicurarsi che gli sviluppatori di software possano usare questa libreria può essere costoso.
    - I processi di sviluppo devono essere adattati per assicurare che la libreria sia usata.

Riuso del software - Introduzione

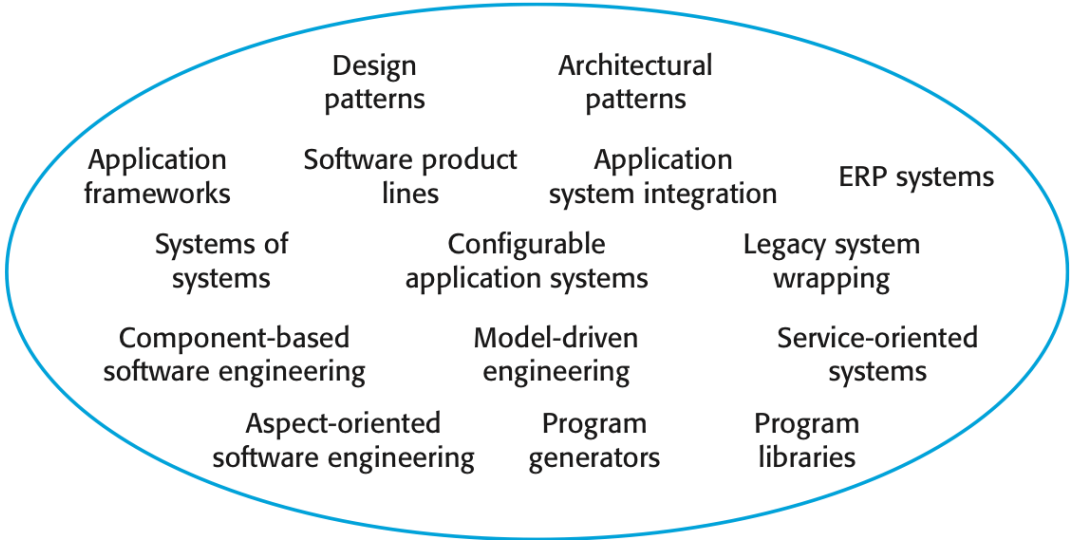
- Trovare, capire e adattare i componenti riutilizzabili:
  - I componenti software devono essere scoperti in una libreria, compresi e, a volte, adattati per funzionare in un nuovo ambiente.
  - Gli ingegneri devono essere ragionevolmente sicuri di trovare un componente nella libreria prima di includere una ricerca di componenti come parte del loro normale processo di sviluppo.
- Aumento dei costi di manutenzione:
  - Se il codice sorgente di un sistema software o di un componente riutilizzato non è disponibile, allora i costi di manutenzione possono essere più alti perché gli elementi riutilizzati del sistema possono diventare sempre più incompatibili con i cambiamenti del sistema.

Riuso del software - Introduzione

- Mancanza di supporto degli strumenti:
  - Alcuni strumenti software non supportano lo sviluppo con il riuso.
  - Può essere difficile o impossibile integrare questi strumenti con un sistema di libreria di componenti.
  - Il processo software assunto da questi strumenti potrebbe non prendere in considerazione il riuso.
  - Questo è particolarmente vero per gli strumenti che supportano l'ingegneria dei sistemi embedded, meno per gli strumenti di sviluppo orientati agli oggetti.
- Sindrome del non inventato qui:
  - Alcuni ingegneri del software preferiscono riscrivere i componenti perché credono di poterli migliorare.
  - Questo ha in parte a che fare con la fiducia e in parte con il fatto che scrivere software originale è visto come più impegnativo che riutilizzare il software di altre persone.

Riuso del software – Il panorama del riuso

- Anche se il riutilizzo è spesso pensato semplicemente come il riutilizzo dei componenti del sistema, ci sono molti approcci diversi al riutilizzo che possono essere usati.
- Il riutilizzo è possibile ad una gamma di livelli, da semplici funzioni a sistemi applicativi completi.
- Il panorama del riuso copre la gamma di possibili tecniche di riuso.
- La seguente figura fornisce una vista del panorama collegato al riuso:



Riuso del software – Il panorama del riuso

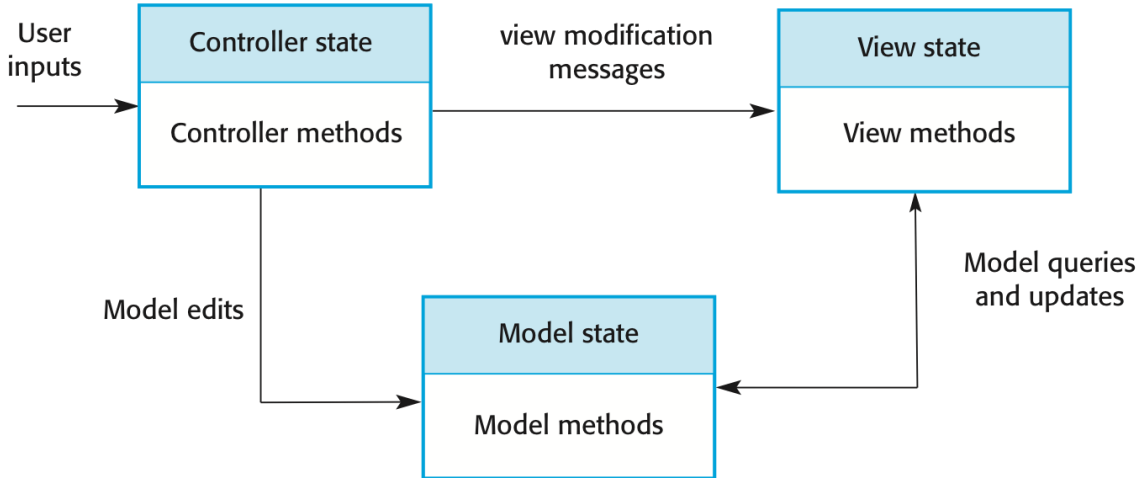
- I fattori chiave da considerare per la pianificazione del riuso sono i seguenti:
  - Il programma di sviluppo del software.
  - La durata prevista del software.
  - Il background, le abilità e l'esperienza del team di sviluppo.
  - La criticità del software e i suoi requisiti non funzionali.
  - Il dominio dell'applicazione.
  - La piattaforma di esecuzione del software.

Riuso del software – Riuso degli application framework

- I framework applicativi sono entità moderatamente grandi che possono essere riutilizzate.
- Sono a metà strada tra il riutilizzo di sistemi e componenti.
- I framework sono un progetto di sotto-sistema composto da una collezione di classi astratte e concrete e le interfacce tra loro.
- Il sottosistema è implementato aggiungendo componenti per riempire parti del progetto e istanziando le classi astratte nel framework.
- I framework per applicazioni web (Web Application Framework - WAF) supportano la costruzione di siti web dinamici come front-end per applicazioni web.
- I WAF sono ora disponibili per tutti i linguaggi di programmazione web comunemente usati, ad esempio Java, Python, Ruby, ecc.
- Il modello di interazione è basato sul modello di progettazione composito Model-View-Controller.
- Un framework MVC supporta la presentazione dei dati in diversi modi e permette l'interazione con ciascuna di queste presentazioni.

Riuso del software – Riuso degli application framework

- Quando i dati vengono modificati attraverso una delle presentazioni, il model del sistema viene cambiato e i controller associati ad ogni view aggiornano la loro presentazione.



- Le principali caratteristiche dei WAF sono le seguenti:
  - Sicurezza: i WAF possono includere classi per aiutare a implementare l'autenticazione dell'utente (login) e l'accesso.
  - Pagine web dinamiche: vengono fornite classi per aiutare a definire modelli di pagine web e a popolarle dinamicamente dal database del sistema.

Riuso del software – Riuso degli application framework

  - Supporto per il database: il framework può fornire classi che forniscono un'interfaccia astratta a diversi database.
  - Gestione delle sessioni: le classi per creare e gestire le sessioni (una serie di interazioni con il sistema da parte di un utente) sono solitamente parte di un WAF.
  - Interazione con l'utente: la maggior parte dei framework web ora fornisce il supporto AJAX, che permette di creare pagine web più interattive.

- I framework sono generici e vengono estesi per creare un'applicazione o un sottosistema più specifico.
- Essi forniscono uno scheletro di architettura per il sistema.
- L'estensione del framework comporta l'aggiunta di classi concrete che ereditano operazioni dalle classi astratte del framework nonchè l'aggiunta di metodi che sono chiamati in risposta ad eventi che sono riconosciuti dal framework.
- Il problema con i framework è la loro complessità, il che significa che ci vuole molto tempo per usarli efficacemente.

Riuso del software – Linee di prodotto software

- Le linee di prodotti software, o famiglie di applicazioni, sono applicazioni con funzionalità generiche che possono essere adattate e configurate per l'uso in un contesto specifico.
- Una linea di prodotti software è un insieme di applicazioni con un'architettura comune e componenti condivisi, con ogni applicazione specializzata per soddisfare requisiti diversi.
- Esempi: un sistema operativo mobile che funziona su diversi modelli di hardware, una linea di software per una famiglia di stampanti con caratteristiche diverse.
- L'adattamento di una linea di software può comportare:
  - La configurazione dei componenti e del sistema.
  - L'aggiunta di nuovi componenti al sistema.
  - La selezione da una libreria di componenti esistenti.
  - La modifica dei componenti per soddisfare nuovi requisiti.

Riuso del software – Linee di prodotto software

- Una linea di prodotti software generalmente include:
  - Componenti di base che forniscono supporto all'infrastruttura.
    - Questi di solito non vengono modificati quando si sviluppa una nuova istanza della linea di prodotti.
  - Componenti configurabili che possono essere modificati e configurati per specializzarli in una nuova applicazione.
    - A volte, è possibile riconfigurare questi componenti senza cambiare il loro codice usando un linguaggio di configurazione dei componenti incorporato.
  - Componenti specializzati, specifici del dominio, alcuni o tutti i quali possono essere sostituiti quando viene creata una nuova istanza di una linea di prodotti.

Riuso del software – Linee di prodotto software

- Le principali differenze tra i framework di applicazioni e le linee di prodotti sono le seguenti:
  - I framework di applicazioni si basano su caratteristiche orientate agli oggetti come il polimorfismo per implementare le estensioni. Le linee di prodotto non hanno bisogno di essere orientate agli oggetti (es. software embedded per un telefono cellulare)
  - I framework di applicazioni si concentrano sulla fornitura di supporto tecnico piuttosto che su quello specifico del dominio. Le linee di prodotto incorporano informazioni sul dominio e sulla piattaforma.
  - Le linee di prodotti spesso controllano applicazioni per apparecchiature.
  - Le linee di prodotti software sono costituite da una famiglia di applicazioni, di solito di proprietà della stessa organizzazione.
- Le architetture delle linee di prodotto devono essere strutturate in modo tale da separare i diversi sotto-sistemi e permetterne la modifica.
- L'architettura dovrebbe anche separare le entità e le loro descrizioni in modo tale che i livelli più alti del sistema accedono alle entità attraverso le descrizioni piuttosto che direttamente.

Riuso del software – Linee di prodotto software

- Possono essere sviluppati vari tipi di specializzazione di una linea di prodotti software:
  - Versioni diverse dell'applicazione sono sviluppate per piattaforme diverse.
  - Diverse versioni dell'applicazione sono create per gestire diversi ambienti operativi, ad esempio diversi tipi di apparecchiature di comunicazione.
  - Versioni diverse dell'applicazione sono create per clienti con diversi requisiti funzionali.
  - Vengono create diverse versioni dell'applicazione per supportare diversi processi di business.
- Le linee di prodotti software sono progettate per essere riconfigurabili. Questa configurazione può avvenire in diverse fasi del processo di sviluppo:
  - Configurazione al momento della progettazione: l'organizzazione che sta sviluppando il software modifica un nucleo comune della linea di prodotti sviluppando, selezionando o adattando i componenti per creare un nuovo sistema per un cliente.
  - Configurazione al tempo di distribuzione: un sistema generico è progettato per essere configurato da un cliente o da consulenti che lavorano con il cliente. La conoscenza dei requisiti specifici del cliente e dell'ambiente operativo del sistema è incorporata nei dati di configurazione che sono usati dal sistema generico.

Riuso del software – Riuso di application system

- Un sistema applicativo è un sistema software che può essere adattato per diversi clienti senza cambiare il codice sorgente del sistema.
- I sistemi applicativi hanno caratteristiche generiche e quindi possono essere usati/riutilizzati in diversi ambienti.
- I sistemi applicativi sono adattati usando meccanismi di configurazione incorporati che permettono di adattare la funzionalità del sistema alle esigenze specifiche del cliente.
- I benefici del riuso dei sistemi applicativi sono i seguenti:
  - Come con altri tipi di riuso, si riesce a realizzare più rapidamente un sistema affidabile.
  - È possibile vedere quali funzionalità sono fornite dalle applicazioni e quindi è più facile giudicare se sono adatte o meno.
  - Alcuni rischi di sviluppo sono evitati usando il software esistente. Tuttavia, questo approccio ha i suoi rischi, come vedremo dopo.
  - Le aziende possono concentrarsi sulla loro attività principale senza dover dedicare molte risorse allo sviluppo di sistemi IT.

Riuso del software – Riuso di application system

- Con l'evoluzione delle piattaforme operative, gli aggiornamenti tecnologici possono essere semplificati perché sono responsabilità del fornitore del prodotto piuttosto che del cliente.

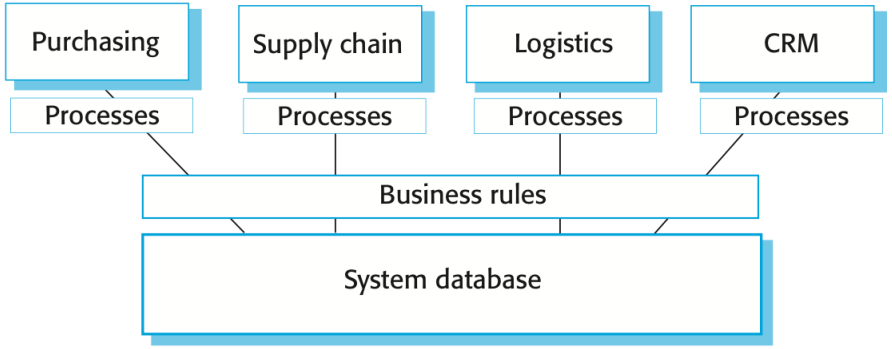
- I problemi di riutilizzo di un sistema applicativo sono i seguenti:
  - I requisiti di solito devono essere adattati per riflettere la funzionalità e il modo di operare del prodotto.
  - Il prodotto può essere basato su presupposti che sono praticamente impossibili da cambiare.
  - Scegliere il giusto sistema per un'impresa può essere un processo difficile, specialmente perché molti prodotti non sono ben documentati.
  - Ci può essere una mancanza di esperienza locale per supportare lo sviluppo dei sistemi.
  - Il fornitore del prodotto controlla il supporto e l'evoluzione del sistema.

- I sistemi applicativi configurabili sono sistemi applicativi generici che possono essere progettati per supportare un particolare tipo di business, un'attività commerciale o, a volte, un'impresa commerciale completa.

- Per esempio, un sistema applicativo può essere prodotto per i dentisti che gestisce gli appuntamenti, i registri dentali, il richiamo dei pazienti, ecc.

Riuso del software – Riuso di application system

- I sistemi specifici del dominio, come i sistemi per supportare una funzione aziendale (ad esempio la gestione dei documenti) forniscono funzionalità che probabilmente saranno richieste da una serie di potenziali utenti.
- Un sistema ERP (Enterprise Resource Planning) è un sistema generico che supporta processi di business comuni come ordini e fatturazione, produzione, etc.
- Questi sono molto usati nelle grandi aziende - rappresentano probabilmente la forma più comune di riuso del software.
- Il nucleo generico viene adattato includendo moduli e incorporando la conoscenza dei processi e delle regole di business.
- L'architettura di un sistema ERP viene mostrata nella seguente figura:



Riuso del software – Riuso di application system

- Gli elementi chiave di una architettura ERP sono i seguenti:
  - Un certo numero di moduli per supportare le diverse funzioni di business.
  - Un insieme definito di processi di business, associato a ciascun modulo, che gestiscono le attività previste da quel modulo.
  - Un database comune che mantiene i dati su tutte le funzioni di business correlate.
  - Un insieme di regole di business che si applicano a tutti i dati del database.

Riuso del software – Sistemi applicativi integrati

- I sistemi applicativi integrati sono applicazioni che includono due o più sistemi applicativi, eventualmente legacy.
- Questo approccio si usa quando non c'è un singolo sistema applicativo che soddisfi tutti i bisogni o quando vogliamo integrare un nuovo sistema applicativo con i sistemi che già usiamo.
- Per sviluppare sistemi applicativi integrati, dobbiamo fare una serie di scelte di progettazione:
  - Quali sistemi applicativi singoli offrono la funzionalità più appropriata? Tipicamente, ci saranno diversi sistemi applicativi disponibili, che possono essere combinati in modi diversi.
  - Come verranno scambiati i dati? Prodotti diversi normalmente usano strutture e formati di dati unici. Dobbiamo scrivere adattatori che convertano da una rappresentazione all'altra.
  - Quali funzionalità di un prodotto saranno effettivamente utilizzate? I singoli sistemi applicativi possono includere più funzionalità di quelle necessarie e le funzionalità possono essere duplicate in prodotti diversi.
- L'integrazione dei sistemi applicativi può essere semplificata se si usa un approccio orientato ai servizi.
- Un approccio orientato ai servizi significa permettere l'accesso alla funzionalità del sistema applicativo attraverso un'interfaccia di servizio standard, con un servizio per ogni unità discreta di funzionalità.

Riuso del software – Sistemi applicativi integrati

- Alcune applicazioni possono offrire un'interfaccia di servizio ma, a volte, questa interfaccia di servizio deve essere implementata dall'integratore di sistema.
- Si deve programmare un wrapper che nasconde l'applicazione e fornisce servizi visibili dall'esterno.
- I principali problemi relativi all'integrazione dei sistemi applicativi sono i seguenti:
  - Mancanza di controllo su funzionalità e prestazioni: i sistemi applicativi possono essere meno efficaci di quanto sembrano.
  - Problemi con l'interoperabilità dei sistemi applicativi: diversi sistemi applicativi possono avere diversi presupposti che rendono difficile l'integrazione.
  - Nessun controllo sull'evoluzione del sistema: i venditori di sistemi applicativi e non gli utenti del sistema controllano l'evoluzione
  - Supporto da parte dei venditori di sistemi: i venditori di sistemi applicativi potrebbero non offrire supporto per tutta la durata del prodotto